CS361 Assignment 1

Tomas Salaz

10 September 2025

Problem 1

A. Write Ascending Algorithm

```
isAscending (A)
if A.length <= 1
    return True
for i = 1 to A.length-1
    if A[i] > A[i+1]
        return False
return True
```

B. Invariant for the Loop

Before each iteration of the loop, the sub array `A[1:i-1]` is ascending.

C. Prove the Loop Invariant

Initialization: Before the First Loop iteration, the trivial case(s) of `A.length`$\leq$`1` is handled and returns true. In the rest of the cases where `A.length`$\geq$`2`, the invariant `A[1:i-1]` is satisfied.
Maintenance: In each iteration of the loop, the sub array, `A[1:i]` is sorted. The current iteration of the loop compares the current and the next element, `A[i]` and `A[i+1]`. If Descending, the loop short circuits to false, If Ascending then loop continues and `A[1:+1]` is now ascending.
Termination: The loop ends when `i=A.length`. According to the Invariant, the array from `A[1:i-1]` is ascending so the array `A[1:A.length]` is Ascending.

D. Prove Correctness

The output of the Algorithm is True if the input Array is ascending, otherwise the algorithm short circuits and outputs False if descending.

Problem 2

A. Write Ascending Algorithm

```
maxValAndOcc (A)
    max = A[1]
    occ = 1
    if A.length = 1
        retuen (max, occ)
    for i = 2 to A.length
        if A[i] = max
            occ = occ + 1
        if A[i] > max
            Max = A[i]
            occ = 1
    return (max, occ)
```

B. Invariant for the Loop

Before each loop iteration, `max` is the max value in the subarray `A[1:i-1]` and `occ` is the number of times the max value has appeared.

C. Prove the Loop Invariant

Initialization: Before the first iteration, we check the special case.
If `A.length=1`, then it is trivial that the max is the value of `A[1]` and there is only one occurrence, so `max=A[1]` and `occ=1`. Otherwise, `max=A[1]` and `occ=1` is already initialized, and the loop begins at `i=2`.
Maintenance: According to the loop invariant, the variables `max` and `occ` are accurate for the sub array `A[1:i-1]`. The current value of the loop `A[i]` is checked with the current `max` and `occ`. After the iteration, the sub array `A[1:i]` would have the correct values and we could go to the next iteration.
Termination: The loop ends when `i=A.length+1`. According to the loop invariant, the sub array `A[1:A.length]` and its `max` and `occ` variables would be correct in the case `i=A.length+1`

D. Prove Correctness

The outputs `max` and `occ` return the maximum value and the number of occurrences in the array `A[1:i]`

Problem 3

A. Algorithm for Matrix Sum

```
sumMatrix (A, r, c)
```

```
sum = 0
for i=1 to r
    for j=1 to c
        sum = sum + A[i][j]
return sum
```

B. Loop Invariants

Inner: Before each iteration, `sum` is the value from the previous rows, `A[1:i-1][j]`, and the previous column of the current row, `A[i][1:j-1]`, hence `sum = A[1:i-1][j] + A[i][1:j-1]` Outer: Before each iteration, `sum` is only the summation of the previous rows, so `sum = A[1:i-1][j]`

C. Prove Loop Invariant for Outer Loop

Initialization: If `r=0` the Matrix is empty and the appropriate sum is returned. Otherwise the base case, `r=1`. When `r=1`, `sum=0` according to the outer loop invariant.
Maintenance: Before the start of each iteration `i`, `sum=A[1:i-1]`. When the loop executes, the current row is added to the `sum`, according to the inner variant then both loops terminate. Upon termination,
`sum=A[1:i][j]` is correct and `i` is incremented. In this case the outer loop variation holds.
Termination: The outer loop terminates when `i=r+1` and according to the invariant the previous rows
`A[1:i][j]` is summed.

D. Prove Correctness

If the matrix is empty, `sum=0`, otherwise the summation of the matrix is complete when the outer loop terminates. When this loop terminates at `i=r+1`, the summation, according both loop invariants is `sum=A[1:i][1:j]`, which is correct.